

Production Code Audit Report

ExamForge AI | Generated: 2026-08-02

Executive Summary

This report presents the findings of a comprehensive production code audit conducted on the ExamForge AI Next.js codebase. The audit examined dead code, unused components, duplicate logic, unused dependencies, and other quality issues that impact maintainability, bundle size, and security posture. A total of 18 findings were identified across 12 categories, with 2 Critical, 6 High, 7 Medium, and 3 Low severity issues. The most impactful findings are: broken role-based middleware guards (route prefixes do not match actual routes), 14+ unused npm packages contributing significant bundle bloat, and 5x duplicated utility functions across dashboard pages.

Severity Distribution

Severity	Count	Key Areas
Critical	2	Middleware RBAC broken, 14+ unused packages
High	6	Duplicate logic, dead API module, dead stores, console.log proliferation
Medium	7	Unused components, hooks, validators, route constants
Low	3	Dead CSS, unused env var, unused type

Critical Findings

1. Role-Based Middleware Guards Are Broken

The middleware defines role-based route prefixes as `/admin`, `/school`, `/teacher`, `/student`, but the actual application routes use `/dashboard/super-admin`, `/dashboard/school-admin`, `/dashboard/teacher`, `/dashboard/student`. This means the `RoleBasedGuard` never fires for any authenticated route, effectively making role-based access control non-functional. Any authenticated user can access any dashboard route regardless of their role.

File: `src/middleware.ts`, lines 45-50

Fix: Update `ROLE_ROUTE_PREFIXES` to match actual routes, or restructure routes to match the guard prefixes.

2. 14+ Unused npm Packages

The following packages are listed in `package.json` dependencies but are never imported anywhere in the `src/` directory. Together they account for significant bundle bloat and an increased security surface area.

Package	Est. Size	Impact
<code>next-auth</code>	Large	Auth library not used; Supabase Auth is used instead
<code>@mdxeditor/editor</code>	~500KB	Rich text editor never imported
<code>react-syntax-highlighter</code>	~150KB	Code highlighting never used
<code>@prisma/client</code> + <code>prisma</code>	Large	Only in unused <code>db.ts</code> ; Supabase is used

Package	Est. Size	Impact
@dnd-kit/core + sortable + utilities	Medium	Drag-and-drop never used
dexie	Medium	IndexedDB wrapper never used
sharp	Large	Image processing never used
next-intl	Medium	i18n never configured
next-pwa	Small	PWA config never set up
react-markdown	Medium	Markdown rendering never used
uuid	Small	UUID generation never used
@reactuses/core	Medium	React hooks library never used
z-ai-web-dev-sdk	Small	AI SDK never used

High Severity Findings

3. 5x Duplicate formatRelativeTime()

The `formatRelativeTime()` utility function is defined identically in 5 separate files: `dashboard/student/page.tsx`, `dashboard/teacher/page.tsx`, `dashboard/school-admin/page.tsx`, `dashboard/super-admin/page.tsx`, and `notifications/page.tsx`. A shared version already exists at `src/lib/utlis/format.ts` but is not imported. This creates maintenance burden and inconsistency risk.

4. 3x Duplicate formatCurrency()

The `formatCurrency()` function is duplicated in `dashboard/school-admin/page.tsx`, `dashboard/super-admin/page.tsx`, and `billing/page.tsx`. A shared version exists at `src/lib/utlis/format.ts`. Each copy has slightly different signatures, creating inconsistency.

5. Entire @/lib/api/ Module Is Dead Code

Four files comprising the API client module (`client.ts`, `errors.ts`, `result.ts`, `index.ts`) are never imported by any page, service, or component. All services use direct Supabase client calls instead. This represents approximately 300+ lines of dead code.

6. Prisma Client Never Used

`src/lib/db.ts` initializes a Prisma client, but it is never imported anywhere. The project uses Supabase exclusively for data access. Both `@prisma/client` and `prisma` are unnecessary dependencies.

7. 3 Unused Zustand Stores

`useUIStore`, `useExamSessionStore`, and `useThemePreferences` are defined in `src/lib/stores/` but never imported by any component or page. The `toastService` wrapper around `sonner` is also never used, as all pages use `sonner` directly. This creates three competing toast/notification systems.

8. 34 Production console.error/warn Statements

There are approximately 34 `console.error` and `console.warn` statements across the codebase, including in all service files, API routes, and the realtime provider. In production, these can leak sensitive data in error messages and provide no structured logging or alerting capability.

9. 20+ Route Constants Point to Non-Existent Pages

The routes constants file defines 20+ route constants for pages that do not exist in the app, including EXAM_TEMPLATES, STUDENT_PRACTICE, STUDENT_FLASHCARDS, STUDENT_AI_TUTOR, ADMIN_SCHOOLS, ADMIN_ANALYTICS, and many more. These orphaned constants create navigation dead-ends and confusion.

Medium Severity Findings

10. 8 Unused Components

QuickActions, RecentActivity, SelectField, TextField, TextareaField, ConfirmDialog, AppShell, and Breadcrumbs are defined in src/components/ but never imported by any page or other component.

11. 3 Unused Custom Hooks

useDebounce, useMediaQuery (with useBreakpoint, useBreakpointBetween, useCurrentBreakpoint), and useOffline are defined in src/lib/hooks/ but never imported.

12. validate.ts and logger.ts Never Used

The entire validate.ts module (emailSchema, urlSchema, phoneSchema, etc.) and the logger.ts module are never imported. The logger is only used by the dead apiClient module.

13. Onboarding Route Guard References Non-Existent Page

The middleware guards the /onboarding route, but no /onboarding page exists in the app. The OnboardingGuard will never redirect users because the page does not exist.

14. Feature Server Actions Never Imported

login.action.ts, logout.action.ts, signup.action.ts, and schools/actions.ts are defined but never imported by any client component. These server actions are dead code.

15. Duplicate Supabase Auth + Profile Fetch Pattern

Multiple pages (results, question-bank, cbt, students, teachers, parents) repeat the exact same pattern of: getUser() + redirect check + profile query + type assertion. This should be extracted into a shared utility function.

Low Severity Findings

The .scrollbar-thin CSS class in globals.css is never applied. The DATABASE_URL environment variable is defined but never referenced in src/. The ExamCategoryType in supabase/types.ts is defined but never imported. These are minor cleanup items.

Recommended Actions

Priority	Action	Impact
P0	Fix middleware RBAC route prefixes	Security: Role-based access control is currently non-functional
P0	Remove 14+ unused packages	Bundle size reduction, security surface reduction
P1	Consolidate duplicate formatRelativeTime/formatCurrency	Maintenance, consistency
P1	Delete dead modules (api/, db.ts, toast-service, validate.Credentials,)	Code clarity, dead code removal
P1	Delete unused components, hooks, and stores	Code clarity, reduced bundle
P1	Replace console.error/warn with structured logging	Observability, security
P2	Clean up route constants for non-existent pages	Navigation accuracy
P2	Create or remove onboarding route	Feature completeness
P2	Extract shared auth+profile utility	Code reuse, consistency